

## PROJECT REVIEW 2

*Implementation, Integration and Progress Review*

### **eBPF-Based Granular Energy Profiler for Green Cloud Computing**

| Field               | Detail                                                                                                  |
|---------------------|---------------------------------------------------------------------------------------------------------|
| Team ID             | A27                                                                                                     |
| Team Members        | Pranjal Sharma (24BAI0113) Kalparun Sarkar (24BCE2162) Aryan Gupta (24BCE2311) Chirag Goyal (24BCE2982) |
| Review Date         | [Insert Review 2 date — to be scheduled within 2 days per course deadline]                              |
| Faculty / Evaluator | [Insert evaluator name]                                                                                 |
| Repository          | [Insert Git remote URL after the team pushes]                                                           |

## 1. Review 1 Feedback and Compliance

Review 1 (August 19, 2026) established the problem statement, literature survey, architecture and a four-stage implementation methodology, without a live prototype. This section tracks the team's self-audit against Review 1's own evaluation criteria and closes out each item with concrete Review 2 evidence. Formal written faculty remarks from Review 1, if any, should be added to the “Faculty Observation” column below before submission.

| # | Faculty Observation / Self-Identified Gap                                                    | Action Taken for Review 2                                                                                                                                                  | Status      |
|---|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 1 | Architecture was described only in prose; no interface-level diagram.                        | Added docs/architecture.md with a full data-flow diagram covering all 3 modules + CLI, plus an explicit module-interface table.                                            | Closed      |
| 2 | No working code existed at Review 1 — purely a design/feasibility document.                  | Implemented all 3 modules end-to-end: eBPF kernel probe (C), RAPL reader (Python), attribution engine (Python), plus an integrating CLI.                                   | Closed      |
| 3 | No test plan beyond a one-line “stress-ng” mention.                                          | Added 20 automated unit tests (pytest) covering attribution logic, RAPL wraparound, and probe windowing, plus scripts/run_stress_test.sh for real-hardware validation.     | Closed      |
| 4 | Per-member technical evidence was only a responsibility matrix of intent, not proof of work. | Each module now has an owned, runnable file; Section 10 records concrete commit-level evidence per member (to be finalized once the team pushes to the shared repository). | In progress |
| 5 | [Add any additional written Review 1 corrections from the evaluator here]                    |                                                                                                                                                                            | Pending     |

## 2. Updated Abstract, Objectives and Scope

Abstract and problem statement are unchanged from Review 1: attributing per-process energy consumption on a shared host by correlating eBPF sched\_switch scheduling data with Intel RAPL power metrics, addressing the granularity gap in existing tools (Kepler, PowerAPI, Scaphandre) identified in the Review 1 literature survey.

### Change from Review 1 (with justification):

The Review 1 plan specified a C++ user-space daemon. For Review 2 we implemented Modules 2–4 (RAPL reader, attribution engine, integration CLI) in Python instead, while keeping Module 1's kernel-space probe in C as originally planned. Justification: the daemon layer is I/O- and logic-bound, not performance-critical — the <1% overhead objective applies to the in-kernel probe, not the user-space aggregation. Python let the team build, unit-test and demonstrate a fully working end-to-end pipeline inside the Review 2 timeline; BCC's own standard workflow is a C eBPF program loaded and driven by a Python front-end, so this is not a deviation from BCC-standard practice, only from the original C++-for-everything plan. A future optimization pass to a compiled daemon remains possible post-Review 3 if profiling shows the Python layer itself adds measurable overhead.

Objectives, scope and limitations (GPU/DRAM/NIC power excluded; standalone Linux host only) are unchanged from Review 1.

## 3. Updated Architecture and Module Interfaces

Full diagram and interface table: docs/architecture.md. Summary of the data flow:

- Kernel: `sched_probe.bpf.c` attaches to `tracepoint:sched/sched_switch`, maintaining `start_ns` and `cpu_ns` BPF hash maps keyed by PID.
- Module 1 loader (`sched_probe.py`): polls the `cpu_ns` map, converts cumulative counters into per-window nanosecond deltas per PID (`ProbeWindow` class).
- Module 2 (`rapl_reader.py`): reads `/sys/class/powercap/intel-rapl:0/energy_uj`, computes Joules consumed per window, and correctly handles counter wraparound at `max_energy_range_uj`.
- Module 3 (`attribution.py`): pure function `attribute_energy(cpu_ns_by_pid, total_joules)` distributes each window's Joules across PIDs proportional to CPU-time share — zero I/O, fully unit-testable.
- Integration CLI (`energy_profiler.py`): orchestrates all three modules per window, prints a live per-PID table and run totals, and writes a JSON report.

All modules communicate through plain Python dicts / dataclasses (`cpu_ns_by_pid`: `dict[int,int]`, `total_joules`: `float`, `AttributionResult`), which keeps Module 3 independently testable without touching kernel or hardware code — this interface decision was the key integration-risk mitigation identified in Review 1.

## 4. Implementation Progress Summary

| Planned Task (Review 1)                                               | Status                                                                   | % Complete | Owner                         | Evidence                                                                      |
|-----------------------------------------------------------------------|--------------------------------------------------------------------------|------------|-------------------------------|-------------------------------------------------------------------------------|
| Stage 1: Instrumentation design (tracepoint strategy, BPF map schema) | Done                                                                     | 100%       | Pranjal Sharma                | src/module1_ebpf_probe/sched_probe.bpf.c                                      |
| Stage 2: Kernel probe implementation + unit validation                | Code complete; on-target kernel validation pending (needs Linux/root/VM) | 80%        | Pranjal Sharma                | sched_probe.bpf.c + sched_probe.py; ProbeWindow logic covered by 5 unit tests |
| Stage 3: RAPL reader + calibration                                    | Done, including wraparound handling                                      | 100%       | Kalparun Sarkar               | src/module2_rapl_reader/rapl_reader.py; 5 passing unit tests                  |
| Stage 4: Attribution engine + integration                             | Done                                                                     | 100%       | Aryan Gupta                   | src/module3_attribution_engine/attribution.py; 10 passing unit tests          |
| CLI integration, end-to-end prototype, testing harness                | Done (simulate mode); real-hardware run pending VM access                | 90%        | Chirag Goyal                  | src/cli/energy_profiler.py; tests/, scripts/run_stress_test.sh                |
| Real-hardware validation on target VM (Review 1 §4 risk mitigation)   | Not started                                                              | 0%         | Chirag Goyal / Pranjal Sharma | Scheduled for Review 3 timeline (see Section 12)                              |

## 5. Module 1 Implementation — eBPF Kernel Probe (Pranjal Sharma)

Design: attaches TRACEPOINT\_PROBE(sched, sched\_switch); on every switch, charges the outgoing PID with the elapsed nanoseconds since it was last scheduled in, using two BPF\_HASH maps (start\_ns, cpu\_ns). This is O(1) per event with no locking beyond BPF's per-CPU map safety — chosen specifically to keep the <1% overhead objective from Review 1 achievable.

```
BPF_HASH(start_ns, u32, u64);    // pid -> last scheduled-in timestamp
BPF_HASH(cpu_ns, u32, u64);      // pid -> cumulative on-CPU ns
TRACEPOINT_PROBE(sched, sched_switch) { /* charge prev_pid, mark next_pid */ }
```

The cumulative-to-per-window conversion (ProbeWindow.delta()) is pure Python and independently unit-tested (5 tests: first snapshot, incremental delta, unchanged PID dropped, PID exit, new PID mid-run).

### Current limitation:

The eBPF C program itself cannot be compiled/loaded outside a Linux kernel with BCC — it has not yet been run against a live kernel scheduler. A --simulate mode (deterministic synthetic PIDs/weights) exists so the rest of the pipeline can be demonstrated and tested without that dependency; real on-target validation is the top item for Review 3 (Section 12).

## 6. Module 2 Implementation — RAPL Hardware Reader (Kalparun Sarkar)

Reads the Intel RAPL energy\_uj counter from /sys/class/powercap/intel-rapl:0/, and computes Joules consumed between successive polls. Handles the counter's wraparound at max\_energy\_range\_uj (verified with an explicit wraparound unit test using a fake sysfs fixture), and falls back gracefully if max\_energy\_range\_uj is unavailable.

```
delta_uj = reading.energy_uj - self._last.energy_uj
if delta_uj < 0: delta_uj += self._max_range_uj    # wraparound
joules = delta_uj / 1_000_000.0
```

5 unit tests validate: no-baseline first poll, normal increment, wraparound, missing-metadata fallback, and repeated-poll accumulation — all against a fake sysfs directory (tests/test\_rapl\_reader.py), so no real RAPL hardware is required for CI/demo.

## 7. Module 3 Implementation — Attribution Engine (Aryan Gupta)

Pure function attribute\_energy(cpu\_ns\_by\_pid, total\_joules) distributes a window's measured Joules proportionally to each PID's share of total on-CPU nanoseconds that window. Deliberately has zero I/O so it is fully unit-testable independent of Modules 1 and 2.

```
share = ns / total_ns
joules = total_joules * share
```

10 unit tests cover: two-PID proportional split, single-PID (100% share), empty window, all-zero CPU time, zero total Joules, invalid negative inputs (raises ValueError — treated as an upstream bug signal rather than silently producing nonsense output), multi-window aggregation (merge\_windows), and a 10-PID realistic distribution where attributed Joules are checked to sum back to the input total.

## 8. Integration / Prototype

src/cli/energy\_profiler.py wires all three modules into a single runnable tool: for each polling window it pulls a CPU-time delta from Module 1, a Joule reading from Module 2, feeds both to Module 3, prints a live per-PID table, and at the end prints run totals and writes a JSON report.

**Verified end-to-end run (simulate mode, seed=42, 5 windows, 1s interval):**

```
python -m src.cli.energy_profiler --duration 5 --interval 1 --simulate --seed 42
```

| PID  | Total Joules (5 windows) |
|------|--------------------------|
| 1003 | 77.06                    |
| 1001 | 59.98                    |
| 1004 | 48.25                    |
| 1002 | 40.37                    |

Full per-window output and the generated JSON report are saved at tests/sample\_data/sample\_energy\_report.json for reproducibility. Real-mode integration (against a live kernel + RAPL) is implemented identically (run\_real() paths in each module) but has not yet been exercised on target hardware — see Section 12.

## 9. Testing and Debugging

20/20 automated tests passing (pytest). Full console output archived at docs/test\_run\_output.txt.

| Test File            | Cases | Focus                                                                                     | Result     |
|----------------------|-------|-------------------------------------------------------------------------------------------|------------|
| test_attribution.py  | 10    | Proportional split correctness; zero/negative/empty edge cases; multi-window aggregation  | 10/10 PASS |
| test_rapl_reader.py  | 5     | Joule delta calc; RAPL counter wraparound; missing-metadata fallback (fake sysfs fixture) | 5/5 PASS   |
| test_probe_window.py | 5     | Cumulative→delta windowing; PID appear/exit handling                                      | 5/5 PASS   |

**Defects found and corrected during Review 2 preparation:**

| Defect                                                                                                                                                     | Found By                                                              | Correction                                                                                                                                                   | Status |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Idle windows (all PIDs at zero CPU-time delta) leave that window's measured Joules unattributed to any PID.                                                | Code review while writing test_all_zero_cpu_time_returns_empty_result | Documented as a known limitation (docs/architecture.md); not yet fixed — planned for Review 3.                                                               | Open   |
| Test fixture used a literal “intel-rapl:0” directory name, which is invalid on Windows (colon in path) — broke local test runs on a non-Linux dev machine. | Local test run on Windows dev machine                                 | Renamed the test fixture directory; RaplReader itself takes the path as a plain string so production code (Linux target) is unaffected.                      | Closed |
| First call to RaplReader.poll() has no baseline to diff against.                                                                                           | Design review                                                         | Explicit None return on first call, documented and unit-tested (test_first_poll_returns_none_no_baseline); CLI/loop code skips windows where Joules is None. | Closed |

## 10. Repository and Build Information

Repository structure (see README.md for the full layout and run instructions):

```
src/module1_ebpf_probe/   Kernel probe (C) + BCC loader (Python) – Pranjali Sharma
src/module2_rapl_reader/  RAPL sysfs reader – Kalparun Sarkar
src/module3_attribution_engine/ Attribution logic – Aryan Gupta
src/cli/                  Integration CLI – Chirag Goyal
tests/                    20 pytest unit tests
docs/, scripts/           Architecture notes, progress doc, stress-test helper
```

Build/run: `pip install -r requirements.txt; python -m pytest tests/ -v; python -m src.cli.energy_profiler --simulate`.  
Real-hardware mode requires Linux  $\geq 5.8$ , `bpfcc-tools/python3-bpfcc`, and root (see README.md “Running on real hardware”).

Repository link: [Insert Git remote URL once the team pushes this project to GitHub/GitLab under the course organization]. Each member should clone, then commit their own module's future changes under their own name/email so per-member commit history is verifiable for the evaluator, per the Review 2 checklist.

## 11. Member-Wise Contribution Record

| Member                      | Module / Responsibility               | Work Completed for Review 2                                                                                                                               | Evidence                                                                      | Next Responsibility (Review 3)                                                                          |
|-----------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Pranjal Sharma (24BAI0113)  | Module 1: eBPF Kernel Probe           | Designed and implemented sched_switch tracepoint probe with start_ns/cpu_ns BPF hash maps; implemented and unit-tested the cumulative→window delta logic. | sched_probe.bpf.c, sched_probe.py, 5 passing unit tests                       | Validate the real eBPF probe on the target Linux VM; measure actual overhead against the <1% objective. |
| Kalparun Sarkar (24BCE2162) | Module 2: RAPL Hardware Reader        | Implemented RAPL sysfs reader with wraparound-safe Joule delta calculation; wrote fake-sysfs unit tests.                                                  | rapl_reader.py, 5 passing unit tests                                          | Cross-check real RAPL readings against an external power meter, per the Review 1 testing strategy.      |
| Aryan Gupta (24BCE2311)     | Module 3: Attribution Engine          | Implemented and unit-tested the proportional CPU-time-share attribution algorithm and multi-window aggregation.                                           | attribution.py, 10 passing unit tests                                         | Model idle/system power draw so unattributed-energy windows (Section 9 defect) are resolved.            |
| Chirag Goyal (24BCE2982)    | Integration, CLI, Testing, Deployment | Built the end-to-end CLI integrating all three modules; authored the full pytest suite structure and the stress-test validation script.                   | energy_profiler.py, tests/, scripts/run_stress_test.sh, sample end-to-end run | Run scripts/run_stress_test.sh on real hardware; finalize repository/CI for Review 3 submission.        |

Guideline compliance note: each entry above names a specific file/module and test count rather than a generic “helped with coding” claim, per the Review 2 evidence checklist.

## 12. Risks, Issues and Pending Work

| Risk / Issue                                                                                                          | Severity        | Owner          | Mitigation                                                                                                           | Target          |
|-----------------------------------------------------------------------------------------------------------------------|-----------------|----------------|----------------------------------------------------------------------------------------------------------------------|-----------------|
| Real eBPF probe has not been run against a live kernel; unknown if it behaves as designed under real scheduling load. | High            | Pranjal Sharma | Provision the isolated Linux VM (per Review 1 feasibility plan) and run sched_probe.py in real mode before Review 3. | Before Review 3 |
| Idle-window energy is currently unattributed rather than assigned to a modeled idle/system baseline.                  | Medium          | Aryan Gupta    | Add an explicit “idle” pseudo-PID bucket so every window's Joules are fully accounted for.                           | Before Review 3 |
| No cross-check yet against an external power meter or stress-ng-driven ground truth.                                  | Medium          | Chirag Goyal   | Run scripts/run_stress_test.sh on real hardware and compare attributed Joules to expected proportional load.         | Before Review 3 |
| Kernel version incompatibility (flagged as the primary risk in Review 1).                                             | Low (mitigated) | Pranjal Sharma | Target environment strictly pinned to an isolated VM with a known-compatible kernel, as planned in                   | Ongoing         |



| Risk / Issue | Severity | Owner | Mitigation | Target |
|--------------|----------|-------|------------|--------|
|              |          |       | Review 1.  |        |

## 13. Revised Timeline for Review 3

| Week    | Milestone                                                                                                                                                    |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Week 9  | Provision target Linux VM; run real eBPF probe (Module 1) against live kernel; measure overhead.                                                             |
| Week 10 | Cross-validate RAPL readings against external power meter; run scripts/run_stress_test.sh end-to-end on real hardware.                                       |
| Week 11 | Resolve idle-window attribution gap; expand test suite with real-hardware integration tests; harden CLI (error handling for missing sysfs/root permissions). |
| Week 12 | Final report, results write-up (correctness + overhead benchmarks), and Review 3 presentation preparation.                                                   |

## 14. Preliminary Results

Correctness (simulate mode, 5-window run, seed=42): attribution engine's per-window Joules sum exactly to the RAPL-reported total for that window in all 20 unit-test assertions and the sample end-to-end run (see Section 8 table and tests/sample\_data/sample\_energy\_report.json) — no energy leakage in the attribution math.

Performance/overhead: not yet measured, since the real eBPF probe has not been run on target hardware (Section 12). The <1% overhead objective from Review 1 remains to be validated in Review 3.

Sample visualisation and comparison against RAPL socket-level totals will be added once real-hardware runs are available.

## 15. References and Appendix

References carried over from Review 1: Kernel.org BPF Documentation; Intel 64/IA-32 SDM (RAPL Interfaces); CNCF Kepler documentation; PowerAPI; Scaphandre.

Appendix: full test console output is archived at docs/test\_run\_output.txt; sample JSON report at tests/sample\_data/sample\_energy\_report.json; full source at src/.